

ALPHA EDTECH | DESAFIO FINAL

GUIA PROFISSIONAL DOS AGENTES

Barbearia Maracá

Manual de apresentação e uso da arquitetura de agentes para planejamento, implementação, handoff e code review.

6 agentes especializados

Ownership de arquivos

Fluxo de handoff e QA

Modelo prático para o Trello

Versão 1.0 | 29 de agosto de 2026

1. Visão executiva

A arquitetura organiza o trabalho em seis agentes especializados. Cada agente representa uma responsabilidade clara, possui limites de arquivos e entrega seu resultado por meio de um handoff verificável.

O objetivo não é criar seis pessoas ou seis sistemas independentes. O objetivo é fazer com que qualquer uso de IA no projeto tenha:

- um responsável definido;
- uma fonte de requisito identificada;
- um escopo de arquivos controlado;
- dependências explícitas;
- validação proporcional ao risco;
- revisão independente antes da conclusão.

Agente	Papel resumido	Principal resultado
Orquestrador	Organiza a demanda e coordena especialidades	Plano com ordem, donos, escopo e gates
Arquiteto e Banco de Dados	Traduz regras aprovadas em modelo relacional	Schema, constraints, migrations e contrato de dados
Backend API e Autorização	Implementa API, regras, autenticação e RBAC	Endpoints seguros, DTOs, serviços e testes
Frontend SPA	Implementa comportamento da SPA em Vanilla TypeScript	Rotas, estado, DOM, formulários e integração Fetch
UI/UX e CSS	Implementa apresentação, acessibilidade e responsividade	CSS nativo, tokens, estados visuais e assets
QA e Code Review	Revisa de forma independente requisitos e riscos	Evidências, achados por severidade e decisão de review

2. Por que essa divisão existe

O produto combina SPA, API REST, banco SQL, autenticação, autorização por papel, agendamentos e dados financeiros. Colocar todas essas responsabilidades em um único agente aumenta o risco de:

- alterar arquivos fora do escopo;
- inventar decisões de arquitetura;
- misturar regra de negócio com interface;
- tratar ocultação no frontend como segurança;
- criar contratos incompatíveis entre banco, API e SPA;
- aprovar o próprio trabalho sem revisão independente.

A separação preserva especialização e rastreabilidade. O fluxo continua pertencendo à equipe humana: agentes apoiam análise e execução, mas decisões de produto, stack, segurança, publicação e aceite final continuam humanas.

3. Arquitetura operacional



Regra central

Uma tarefa pode envolver vários agentes, mas cada arquivo deve possuir um único escritor por etapa. Se dois agentes precisarem do mesmo arquivo, o Orquestrador define uma sequência e um handoff, nunca uma edição concorrente.

Ordem típica entre domínios

1. Banco define entidades, invariantes e constraints quando o modelo de dados muda.
2. Backend implementa o contrato HTTP e a autorização com base no modelo aprovado.
3. Frontend integra os endpoints e implementa estados e navegação.
4. UI/UX aplica apresentação, responsividade e acessibilidade sobre a estrutura funcional.
5. QA revisa o conjunto, executa os comandos existentes e verifica cenários positivos e negativos.

Nem toda tarefa precisa passar por todos os agentes. Uma correção exclusivamente visual, por exemplo, pode envolver UI/UX, Frontend para revisão de integração e QA.

4. Catálogo dos agentes

4.1 Orquestrador

Missão: transformar uma demanda em um plano executável, selecionar os especialistas corretos, ordenar dependências e impedir expansão indevida de escopo.

Acione quando:

- o card afeta mais de um domínio;
- existe conflito entre documentos, código e decisão da equipe;
- não está claro quem deve alterar cada arquivo;
- há dependência entre schema, API, frontend ou visual;
- uma decisão de produto ou tecnologia ainda não foi aprovada.

Responsabilidades principais:

- confirmar objetivo, fonte, critérios de aceite e restrições;
- nomear um dono para cada entrega;
- definir ordem, caminhos permitidos e validações;
- coordenar contratos e handoffs;
- encaminhar toda implementação para revisão independente.

Não deve: implementar funcionalidade, escolher tecnologia ausente por conveniência ou resolver silenciosamente uma dúvida de produto.

Entrega esperada: mapa de execução com agentes, ordem, arquivos exclusivos, dependências, gates, bloqueios e próximo responsável.

Arquivo de referência: [agents/orchestrator.md](#).

4.2 Arquiteto e Banco de Dados

Missão: traduzir regras aprovadas em uma arquitetura de domínio e em um modelo SQL relacional consistente, normalizado e evolutivo.

Acione quando:

- uma entidade, campo, relacionamento ou status precisa mudar;
- a tarefa exige constraint, índice, transação ou migration;
- existe risco de dupla reserva ou inconsistência concorrente;
- Backend e Frontend precisam de um contrato de dados comum.

Responsabilidades principais:

- modelar entidades e cardinalidades;
- definir chaves, FKs, unicidade, nulabilidade e índices;
- proteger integridade de horários e agendamentos;
- produzir migrations reproduzíveis;

- comunicar impactos ao Backend, Frontend e QA.

Não deve: criar endpoints, telas ou permissões inexistentes; escolher banco, provider ou ORM sem decisão registrada.

Entrega esperada: modelo, alterações de schema, constraints, migration, riscos, compatibilidade e cenários de integridade.

Arquivo de referência: [agents/architect-database.md](#).

4.3 Backend API e Autorização

Missão: implementar a API REST em Node.js e TypeScript, aplicando regras de negócio, validação, autenticação e autorização no servidor.

Acione quando:

- um endpoint ou DTO precisa ser criado ou alterado;
- uma regra de negócio deve ser aplicada no servidor;
- há autenticação, sessão, token, papel ou propriedade de recurso;
- a API precisa consultar, gravar ou agregar dados;
- um cenário de acesso indevido precisa ser impedido.

Responsabilidades principais:

- separar controllers, services, repositories e DTOs;
- validar toda entrada externa;
- aplicar RBAC e propriedade de recurso com negação por padrão;
- proteger dados pessoais, senhas e informações financeiras;
- tratar agendamento de forma atômica contra conflitos;
- documentar contratos para o Frontend e cenários para QA.

Não deve: editar migrations, implementar SPA ou inventar framework, endpoint, papel ou fluxo.

Entrega esperada: rota, método, autenticação, papéis, request, response, erros, efeitos no banco, concorrência e evidências de teste.

Arquivo de referência: [agents/backend-api-security.md](#).

4.4 Frontend SPA

Missão: implementar o comportamento da SPA em Vanilla TypeScript, incluindo roteamento, estado, DOM, formulários e integração com a API.

Acione quando:

- uma rota client-side ou tela funcional precisa mudar;
- há estado, eventos, formulários ou manipulação do DOM;
- a SPA precisa consumir a API via Fetch;
- é necessário tratar loading, vazio, erro, 401, 403 ou falha de rede;
- existe risco de listeners duplicados ou requisições obsoletas.

Responsabilidades principais:

- navegar sem recarregar a página;
- sincronizar URL e estado;
- implementar componentes de DOM sem framework;
- consumir contratos tipados;
- apresentar validações e erros do backend;
- manter foco, teclado e semântica acessíveis.

Não deve: tratar guarda visual como autorização real, alterar schema ou definir o visual final sem coordenação com UI/UX.

Entrega esperada: rotas, estados, eventos, chamadas de API, tipos, erros, hooks visuais e passos de validação.

Arquivo de referência: [agents/frontend-spa.md](#).

4.5 UI/UX e CSS

Missão: construir uma interface responsiva, acessível e coerente usando CSS3 nativo e microinterações performáticas.

Acione quando:

- a tarefa envolve layout, responsividade, tokens ou componentes visuais;
- é necessário implementar hover, foco, disabled, loading, vazio ou erro;
- existem assets, animações, contraste ou requisitos de teclado;
- uma tela funcional precisa receber acabamento visual.

Responsabilidades principais:

- definir variáveis CSS para cor, espaço, tipografia, raio e movimento;
- organizar CSS modular com Flexbox e Grid;
- representar todos os estados do fluxo;
- respeitar foco visível e [prefers-reduced-motion](#);
- validar desktop, mobile, contraste e alvos de toque;
- otimizar apenas assets aprovados.

Não deve: implementar regra de negócio, Fetch, endpoint, schema ou adotar Tailwind, Bootstrap ou CSS-in-JS.

Entrega esperada: componentes, tokens, breakpoints, estados, animações, assets, requisitos de markup e checklist de acessibilidade.

Arquivo de referência: [agents/ui-ux-css.md](#).

4.6 QA e Code Review

Missão: verificar de forma independente se a entrega atende requisitos, segurança, tipagem, contratos e experiência, sem corrigir o código de produção no lugar do proprietário.

Acione quando:

- uma implementação está pronta para revisão;

- o diff precisa ser comparado com critérios de aceite;
- autorização, concorrência ou integridade precisam de teste negativo;
- a SPA precisa ser validada em navegação e acessibilidade;
- uma correção retornou para revalidação.

Responsabilidades principais:

- derivar uma matriz de testes do requisito;
- revisar o diff completo e os handoffs;
- executar somente comandos que realmente existem;
- testar sucesso, erro, acesso negado e propriedade de recurso;
- classificar achados por severidade;
- devolver correções ao dono e revalidar depois.

Não deve: alterar código de produção, redefinir requisito, inventar ferramenta ou aprovar falha crítica.

Entrega esperada: resultado do review, comandos executados, cenários, achados, evidências, limitações, responsáveis pelas correções e risco residual.

Arquivo de referência: [agents/qa-code-review.md](#).

5. Matriz de propriedade dos arquivos

Caminho	Dono primário	Participação obrigatória
AGENTS.md e agents/**	Orquestrador	Alteração somente por tarefa explícita
backend/prisma/**	Arquiteto e Banco	Backend revisa impacto; QA valida
backend/src/**	Backend API e Autorização	Banco fornece contratos; QA revisa
frontend/src/** - lógica	Frontend SPA	UI/UX fornece apresentação; QA revisa
frontend/index.html	Frontend SPA	Mudança visual exige coordenação com UI/UX
frontend/public/assets/**	UI/UX e CSS	Frontend referencia os assets
shared/types/**	Escritor nomeado no plano	Banco, Backend e Frontend aprovam o contrato
Testes futuros	QA e Code Review	Caminho depende do runner aprovado

Manifests, infraestrutura, lint, testes, build e deploy ainda precisam de decisões formais antes de receber propriedade definitiva.

6. Fluxo obrigatório de trabalho

Etapa 1 - Intake

O Orquestrador confirma card, objetivo, fonte do requisito, critérios de aceite, prioridade, restrições e caminhos permitidos.

Etapa 2 - Análise

O especialista identifica regras, arquivos, riscos, dependências e validações necessárias. Nenhuma tecnologia ausente deve ser assumida.

Etapa 3 - Decisão

Questões de produto, stack, permissão, segurança, financeiro ou arquitetura voltam ao responsável humano. A resposta deve ser registrada no card ou documento oficial.

Etapa 4 - Implementação

Cada entrega possui um agente responsável e um único escritor por arquivo. Mudanças devem permanecer limitadas aos caminhos autorizados.

Etapa 5 - Validação do responsável

O agente executa typecheck, build, lint, testes ou verificações manuais que estiverem realmente configurados e registra qualquer limitação.

Etapa 6 - Code Review

QA compara requisito, diff, contratos e evidências. Segurança deve incluir cenários negativos, não apenas o fluxo feliz.

Etapa 7 - Correção e revalidação

Achados retornam ao agente proprietário. QA não corrige produção; ele revalida a solução e a regressão relevante.

Etapa 8 - Conclusão

O card só segue para Concluído quando os critérios de aceite possuem evidência, não existem achados críticos abertos e riscos residuais estão documentados.

7. Uso recomendado no Trello

O fluxo oficial do projeto é:

Backlog -> Em Análise/IA -> Em Desenvolvimento -> Code Review -> Concluído

Modelo de card

Título:
História ou descrição:
Objetivo:
Fonte do requisito:
Critérios de aceite:
Agente responsável:
Caminhos permitidos:
Dependências e handoffs:
Decisões já aprovadas:
Validações esperadas:
Responsável humano:
Resumo do prompt de IA utilizado:

Checklist para mover para Em Desenvolvimento

- fonte do requisito identificada;
- critérios de aceite verificáveis;
- agente e responsável humano definidos;
- caminhos permitidos listados;
- dependências resolvidas ou ordenadas;
- decisões ainda pendentes marcadas como bloqueio.

Checklist para mover para Code Review

- handoff preenchido;
- diff limitado ao escopo;
- contratos atualizados e comunicados;
- validações executadas com resultado;
- limitações e riscos registrados;
- nenhum segredo ou dado sensível no card, código ou log.

Checklist para mover para Concluído

- QA registrou o resultado;
- P0 e P1 inexistentes ou resolvidos;
- correções foram revalidadas;
- critérios de aceite possuem evidências;
- decisão humana existe para qualquer risco aceito;
- PR foi revisado e integrado conforme a convenção Git aprovada.

8. Contrato padrão de handoff

Use este bloco ao entregar trabalho entre agentes ou integrantes:

```
Status: concluído | parcial | bloqueado  
Resumo:  
Requisitos atendidos:  
Arquivos alterados:  
Contratos ou schema afetados:  
Validações executadas e resultado:  
Riscos ou dividas introduzidas:  
Decisões pendentes:  
Próximo agente ou dono:  
Registro curto do uso de IA para o card:
```

Um handoff incompleto não autoriza o próximo responsável a adivinhar contrato, permissão ou regra de produto.

9. Exemplos de roteamento de tarefas

Exemplo A - Criar agendamento

1. Orquestrador confirma regra, papéis, aceite e sequência.
2. Arquiteto e Banco define a proteção contra conflito de horário.
3. Backend implementa endpoint, validação, autorização e transação.
4. Frontend implementa formulário, estados e consumo da API.
5. UI/UX aplica calendário, feedbacks, foco e responsividade.
6. QA testa sucesso, entrada inválida, horário ocupado, usuário indevido, rede e acessibilidade.

Exemplo B - Alterar apenas a aparência de um botão

1. Orquestrador pode classificar diretamente como tarefa visual simples.
2. UI/UX altera CSS ou asset no caminho autorizado.
3. Frontend revisa se markup ou eventos foram impactados.
4. QA verifica estados, teclado, contraste e regressão visual.

Exemplo C - Adicionar indicador financeiro

1. Orquestrador confirma que o requisito foi aprovado e é exclusivo do Administrador.
2. Banco avalia dados, precisão monetária e consulta agregada.
3. Backend aplica autorização no servidor e evita vazamento no retorno.
4. Frontend exibe apenas na área administrativa.
5. UI/UX representa valores e estados de forma acessível.
6. QA tenta acessar com Cliente, Barbeiro e Recepcionista antes de aprovar.

10. Regras críticas que todos devem conhecer

- O Backend é a barreira real de autorização; esconder um botão no frontend não protege dados.
- Cliente opera apenas os próprios dados e agendamentos.
- Barbeiro opera apenas a própria agenda e vê somente dados necessários ao atendimento.
- Recepcionista possui acesso operacional, mas não financeiro.
- Informações financeiras são exclusivas do Administrador.
- O sistema deve impedir dupla reserva e horário indisponível.
- Vanilla TypeScript, HTML5 e CSS3 nativo são requisitos; frameworks de frontend e CSS estão proibidos.
- TypeScript deve permanecer estrito; **any** exige exceção humana explícita.
- Exemplos e sugestões dos documentos não se tornam requisitos automaticamente.
- Nenhum agente pode inventar stack, comando, permissão ou resultado de teste.
- Push, publicação, merge e alterações externas exigem autorização humana.
- Toda implementação precisa de revisão independente.

11. Decisões ainda pendentes da equipe

Antes de implementar a base técnica, a equipe precisa registrar decisões sobre:

- banco SQL e provider;
- uso e versão do Prisma;
- framework HTTP do Node;
- gerenciador de pacotes e comandos padrão;
- autenticação, sessão ou token e expiração;
- hashing de senha;
- estrutura de testes, lint, formatação e build;
- recuperação de senha;
- histórico de clientes para Recepcionista;
- política de despesas, lucro e origem dos dados financeiros;
- convenção Git, responsáveis por aprovação, merge e deploy.

Esses pontos são bloqueios conhecidos, não falhas dos agentes. Registrar a decisão uma vez evita que cada tarefa adote uma solução diferente.

12. Roteiro de apresentação para a equipe

Use este resumo em uma reunião rápida:

1. Temos seis agentes, cada um com missão e limites claros.
2. O Orquestrador recebe o card e define quem faz o quê.

3. Banco, Backend, Frontend e UI/UX implementam apenas seus domínios.
4. Um arquivo possui um único escritor por etapa.
5. Contratos são entregues por handoff, sem adivinhação entre áreas.
6. QA revisa de forma independente e devolve correções ao dono.
7. PRD, especificação e decisões humanas prevalecem sobre inferências da IA.
8. Nenhuma tarefa termina sem critérios de aceite, evidências e revisão.

13. Referências internas

- Manual central: [AGENTS.md](#)
- Orquestração: [agents/orchestrator.md](#)
- Arquitetura e banco: [agents/architect-database.md](#)
- Backend e segurança: [agents/backend-api-security.md](#)
- Frontend SPA: [agents/frontend-spa.md](#)
- UI/UX e CSS: [agents/ui-ux-css.md](#)
- QA e Code Review: [agents/qa-code-review.md](#)

14. Manutenção do guia

Atualize este documento quando houver mudança aprovada na divisão de responsabilidades, na estrutura de pastas ou no fluxo de revisão. Mudanças normativas devem ser feitas primeiro em [AGENTS.md](#) e no brief do agente correspondente; este guia deve então ser sincronizado para continuar sendo uma apresentação fiel da arquitetura.